



Design and Development of a Smart Vision Assistance System for Visually Impaired Individuals

Summer Internship Project

Submission Date: 16/07/2026

Submitted By: Riddhima Jaiswal

College Name: University Institute of Engineering and Technology Panjab
University Chandigarh

Under the Supervision of:

Dr. Thanjai L. Oppili Prasad

**Department of Electronics Engineering Indian
Institute of Technology (BHU), Varanasi**

Content

Abstract

1. Introduction
2. System Overview
3. System Implementation
4. System Flowcharts
 - 4.1. Prototype-I: ESP32-CAM Based System Architecture
 - 4.2. Prototype-II: Raspberry Pi Camera Module Processing Flow
5. Proposed Wearable Hardware Prototypes
 - 5.1. Prototype-I: ESP32-CAM Based Wearable Smart Glasses
 - 5.2. Prototype-II: Raspberry Pi Camera Based Wearable System
 - 5.3. Discussion
6. Implementation Results
 - 6.1. Depth Estimation Model
7. Experimental Validation
8. Experimental Performance Evaluation
 - 8.1. Performance Evaluation of Raspberry Pi Camera Prototype
 - 8.2. Stereo Depth Estimation Evaluation
 - 8.3. OCR Performance Evaluation
 - 8.4. ESP32-CAM Wireless Prototype Performance
 - 8.5. Wireless Communication Characteristics
 - 8.6. Comparison Between the Two Prototypes
9. Current Limitations
10. Future Work
11. Conclusions

References

Design and Development of a Smart Vision Assistance System for Visually Impaired Individuals

Department of Electronics Engineering
Indian Institute of Technology (BHU), Varanasi

July 14, 2026

Abstract

This document presents the design and development of a Smart Vision Assistance System intended to improve the independence and mobility of visually impaired individuals. The proposed wearable system integrates real-time object detection, optical character recognition (OCR), stereo vision-based depth estimation, and voice feedback into a single portable platform. Two hardware prototypes were developed to evaluate different implementation approaches while maintaining a common software framework. The objective of this documentation is to describe the overall system architecture, hardware implementation, software workflow, experimental validation, design limitations, and possible future improvements.

1 Introduction

Visual impairment significantly affects a person's ability to navigate unfamiliar environments, identify surrounding objects, and access printed information. Traditional assistive devices such as white canes and guide dogs provide limited environmental awareness and cannot recognize objects or read textual information in real time.

Recent advancements in embedded systems, computer vision, and artificial intelligence have enabled the development of wearable assistive technologies capable of providing intelligent environmental perception. In this work, a Smart Vision Assistance System has been developed by integrating stereo vision, object detection, optical character recognition, and speech synthesis into a compact wearable platform.

The primary objective of the proposed system is to assist visually impaired users by detecting nearby objects, estimating their distance, recognizing printed text, and communicating the information through audio feedback. The system has been implemented using embedded computing hardware and open-source computer vision frameworks to achieve a cost-effective and portable solution.

2 System Overview

The Smart Vision Assistance System is a wearable embedded vision platform designed to assist visually impaired individuals through real-time environmental perception. The

proposed system integrates stereo vision, artificial intelligence, optical character recognition, and voice feedback to provide meaningful information about surrounding objects and textual content.

Two hardware implementations have been developed during the course of this work.

- Prototype-I: Dual ESP32-CAM modules communicating wirelessly with a Raspberry Pi 5.
- Prototype-II: Dual Raspberry Pi Camera Modules directly connected to the Raspberry Pi 5 through CSI interfaces.

Both implementations share a common software framework while differing only in their image acquisition architecture.

3 System Implementation

The developed system consists of the following major software modules.

1. Stereo Camera Initialization

Both cameras are initialized simultaneously to capture synchronized stereo image pairs.

2. Stereo Calibration

Camera intrinsic and extrinsic parameters are loaded from the calibration file for stereo image correction.

3. Stereo Rectification

The captured images are rectified to align corresponding epipolar lines prior to disparity computation.

4. Depth Estimation

Stereo Semi-Global Block Matching (SGBM) is used to generate disparity maps, followed by 3D reconstruction for object distance estimation.

5. Object Detection

YOLO-based deep learning models perform real-time object detection and classification.

6. Optical Character Recognition

EasyOCR recognizes textual information from the captured scene.

7. Audio Feedback

The detected object names, distances, and recognized text are converted into speech using the Text-to-Speech engine.

8. Multi-threaded Execution

Separate background threads are used for camera acquisition, OCR processing, and speech synthesis to improve real-time performance.

4 System Flowcharts

To illustrate the operational workflow of the proposed Smart Vision Assistance System, separate flowcharts are presented for the two hardware architectures considered in this work. Although both prototypes implement the same computer vision pipeline, they differ in their image acquisition hardware and communication mechanisms.

The first architecture employs dual ESP32-CAM modules for wireless stereo image acquisition, whereas the second architecture utilizes dual Raspberry Pi Camera Modules directly connected to the Raspberry Pi 5 through CSI interfaces. The corresponding processing workflows are described below.

4.1 Prototype-I: ESP32-CAM Based System Architecture

Figure 1 illustrates the processing workflow of the proposed ESP32-CAM based stereo vision system. The left and right ESP32-CAM modules simultaneously acquire stereo images and transmit synchronized video streams wirelessly to the Raspberry Pi 5. The acquired frames undergo stereo calibration, image rectification, disparity estimation, object detection using YOLO, text recognition using EasyOCR, information fusion, and finally audio feedback generation for the visually impaired user.

4.2 Prototype-II: Raspberry Pi Camera Module Processing Flow

Figure 2 presents the complete processing flow of the Raspberry Pi Camera based implementation. After system initialization, synchronized stereo images are captured through the dual Raspberry Pi Camera Modules. The acquired frames are rectified before disparity estimation using the StereoSGBM algorithm. Subsequently, YOLO performs object detection while EasyOCR recognizes scene text. The extracted object information, estimated depth, and recognized text are fused together to generate real-time voice guidance for the user.

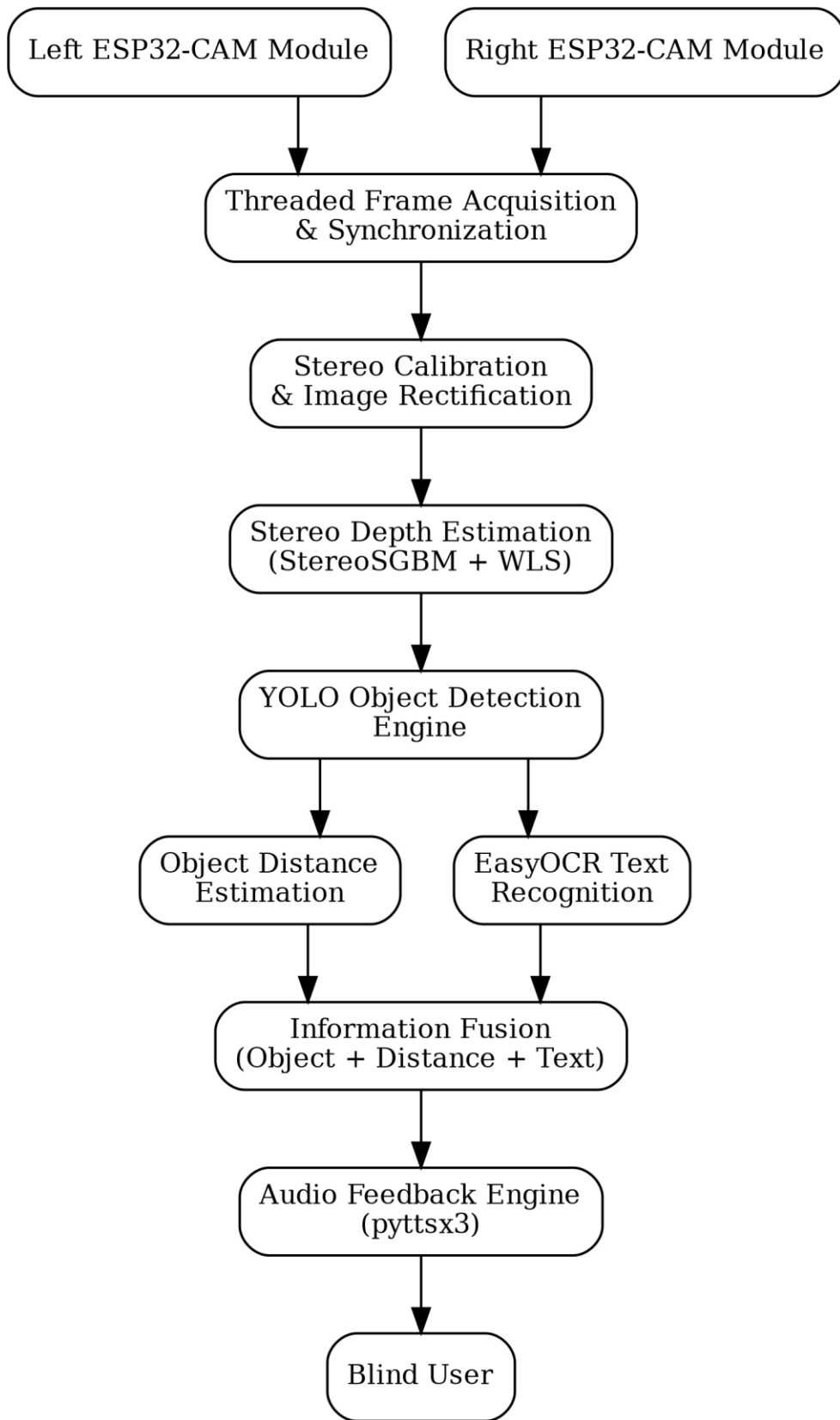


Figure 1: Overall processing architecture of the proposed ESP32-CAM based Smart Vision Assistance System.

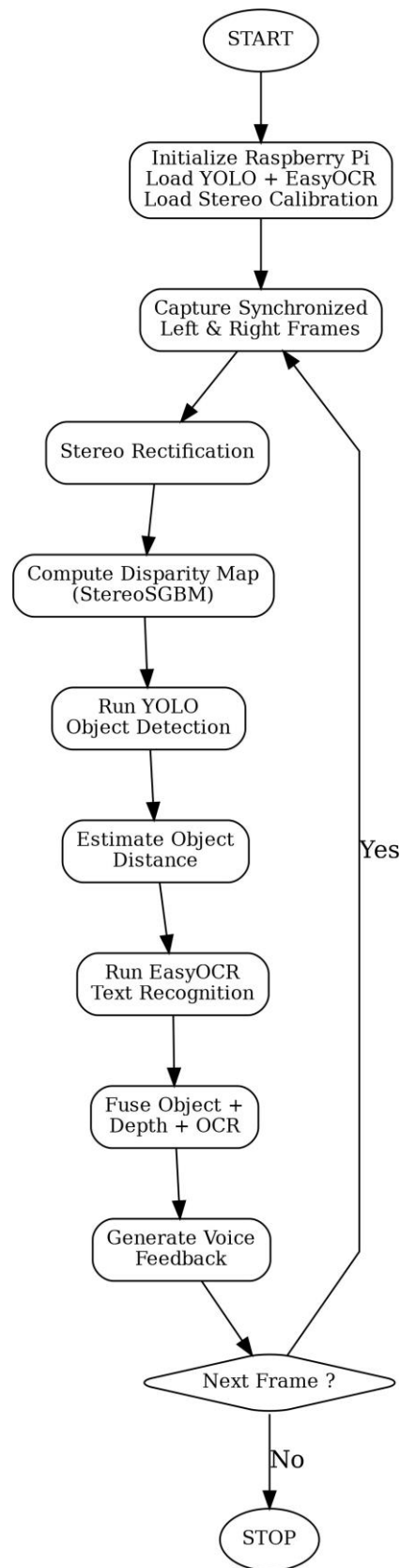


Figure 2: Execution flowchart of the proposed Raspberry Pi Camera Module based Smart Vision Assistance System.



Figure 3: Proposed wearable smart glasses prototype based on dual ESP32-CAM modules and Raspberry Pi 5 processing.

5 Proposed Wearable Hardware Prototypes

To improve the portability and practical usability of the proposed Smart Vision Assistance System, two wearable hardware prototype designs are proposed. Both prototypes implement the same computer vision pipeline while employing different stereo image acquisition hardware. The first prototype utilizes dual ESP32-CAM modules for wireless image acquisition, whereas the second prototype employs dual Raspberry Pi Camera Modules connected directly to a Raspberry Pi 5 through CSI interfaces. These proposed designs emphasize lightweight construction, user comfort, modularity, and ease of deployment for visually impaired users.

5.1 Prototype-I: ESP32-CAM Based Wearable Smart Glasses

The first proposed prototype is based on dual ESP32-CAM modules mounted on a wearable smart glasses frame. The two camera modules are positioned at a fixed stereo baseline to acquire synchronized left and right images. Each ESP32-CAM module is powered using a compact rechargeable battery pack mounted behind the user's head, thereby eliminating long power cables around the face.

The computationally intensive computer vision algorithms, including stereo depth estimation, object detection, and optical character recognition, are executed on a Raspberry Pi 5 carried separately inside a lightweight waist pouch. This architecture minimizes the weight of the head-mounted device while providing a wireless stereo vision platform suitable for assistive applications.



Figure 4: Proposed wearable Raspberry Pi stereo vision prototype with dual Raspberry Pi Camera Modules and portable processing enclosure.

5.2 Prototype-II: Raspberry Pi Camera Based Wearable System

The second proposed prototype utilizes dual Raspberry Pi Camera Modules mounted on a rigid stereo camera bar with a fixed baseline of approximately 127 mm. Both cameras are connected directly to the Raspberry Pi 5 through a dual CSI interface, enabling synchronized image acquisition with low communication latency.

The Raspberry Pi 5 together with a portable power bank is enclosed inside a compact ventilated enclosure that can be comfortably carried by the user. The enclosure is designed to provide mechanical protection, effective heat dissipation, and continuous battery-powered operation while maintaining system portability.

5.3 Discussion

The ESP32-CAM based prototype offers a lightweight wireless architecture that reduces the amount of cabling around the user's head and improves overall wearing comfort. In contrast, the Raspberry Pi Camera based prototype provides direct high-speed CSI communication, resulting in improved synchronization and lower image acquisition latency. Both proposed wearable designs are intended to serve as practical platforms for implementing real-time object detection, stereo depth estimation, optical character recognition, and audio-based navigation assistance for visually impaired users.

6 Implementation Results

The developed software framework was successfully implemented on two hardware configurations: (i) a wireless stereo vision system using dual ESP32-CAM modules and Raspberry Pi 5, and (ii) a direct stereo vision system using dual Raspberry Pi Camera Modules interfaced with Raspberry Pi 5. Both implementations successfully integrate stereo vision, object detection, optical character recognition, and voice feedback within a unified software framework.

Table 1 summarizes the implemented features.

Table 1: Implementation Summary

Feature	ESP32 Prototype	Raspberry Pi Prototype
Stereo Image Acquisition	Yes	Yes
Stereo Calibration	Yes	Yes
Stereo Rectification	Yes	Yes
Stereo Depth Estimation	Yes	Yes
YOLO Object Detection	Yes	Yes
EasyOCR Text Recognition	Yes	Yes
Audio Feedback	Yes	Yes
Headless Mode	Yes	Yes
Display Mode	Yes	Yes
Multi-threaded Processing	Yes	Yes
Wireless Image Streaming	Yes	No
CSI Camera Interface	No	Yes

Table 2: Comparison Between the Two Proposed Prototypes

Parameter	ESP32 Prototype	Raspberry Pi Prototype
Camera Module	ESP32-CAM	Raspberry Pi Camera Rev1.3
Communication	Wi-Fi Streaming	CSI Interface
Image Resolution	640 × 480	640 × 480
Object Detection	YOLOv8 Nano	YOLO11 Nano
OCR Engine	EasyOCR	EasyOCR
Depth Estimation	StereoSGBM + WLS	StereoSGBM
Processing Unit	Raspberry Pi 5	Raspberry Pi 5
Voice Feedback	pyttsx3	pyttsx3
Display Support	Yes	Yes
Headless Execution	Yes	Yes
Image Synchronization	Software	Hardware
Latency	Moderate	Low

6.1 Depth Estimation Model

The object distance is estimated using the standard stereo vision relationship,

$$Z = \frac{fB}{d} \quad (1)$$

Table 3: System Configuration and Processing Parameters

Parameter	Value	Description
Image Resolution	640 × 480 pixels	Stereo image acquisition
Stereo Baseline (RPI Prototype)	127 mm	Camera center-to-center distance
Stereo Baseline (Code)	60 mm	Depth calculation constant
YOLO Confidence Threshold	0.50–0.60	Object detection threshold
Stereo Matching Algorithm	StereoSGBM	Disparity estimation
Number of Disparities	64 / 96	Stereo matching search range
Stereo Block Size	5 pixels	Matching window size
OCR Interval	10 frames	Text recognition frequency
Frame Skip	5 frames	Object detection interval
Speech Rate	145–160 WPM	Text-to-speech output
Input Image Channels	RGB (3-channel)	Camera input
Depth Calculation	$Z = \frac{fB}{d}$	Stereo depth equation
Processing Platform	Raspberry Pi 5	Main processing unit
OCR Engine	EasyOCR	Text recognition
Object Detection Engine	YOLOv8n / YOLO11n	Deep learning detector

where

- Z = Estimated object distance (m)
- f = Camera focal length (pixels)
- B = Stereo baseline (m)
- d = Measured disparity (pixels)

For the Raspberry Pi implementation, the software uses

$$B = 0.06 \text{ m}, \quad f = 700 \text{ pixels}$$

which gives

$$Z = \frac{700 \times 0.06}{d} = \frac{42}{d} \text{ meters.}$$

For example,

$$d = 20 \text{ pixels}$$

gives

$$Z = \frac{42}{20} = 2.1 \text{ m.}$$

Similarly,

$$d = 40 \text{ pixels}$$

gives

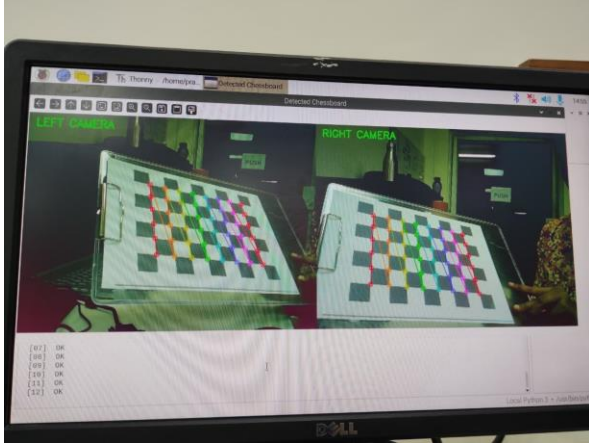
$$Z = \frac{42}{40} = 1.05 \text{ m.}$$

7 Experimental Validation

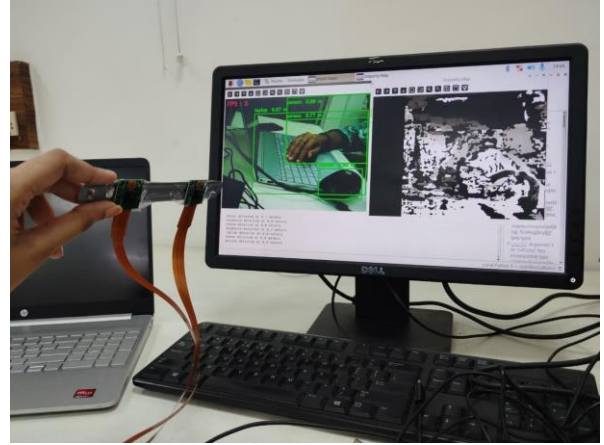
The proposed Smart Vision Assistance System was experimentally validated using both the Raspberry Pi Camera based stereo vision system and the ESP32-CAM based wireless stereo vision system. The experiments verified stereo camera calibration, disparity map generation, stereo depth estimation, object detection, optical character recognition (OCR), and real-time information fusion. Figure 5 presents the major experimental demonstrations carried out during the implementation stage.

Table 4: Design and Mathematical Parameters of the Proposed Smart Vision Assistance System

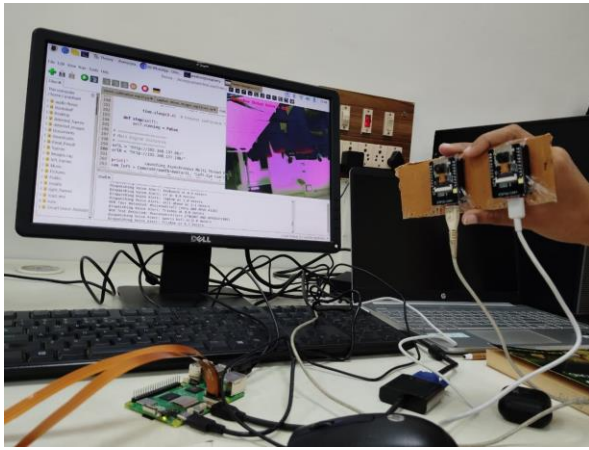
Parameter	ESP32-CAM Prototype	Raspberry Pi Camera Prototype
Stereo Camera Configuration	Dual ESP32-CAM	Dual Raspberry Pi Camera Rev 1.3
Image Resolution	640 × 480 pixels	640 × 480 pixels
Stereo Baseline (B)	110 mm (Wearable Design)	127 mm (Camera Bar)
Effective Baseline Used in Software	Calibration Matrix	60 mm (0.06 m)
Image Channels	RGB (24-bit)	RGB (24-bit)
Object Detection Model	YOLOv8 Nano	YOLO11 Nano
OCR Engine	EasyOCR	EasyOCR
Stereo Matching	StereoSGBM + WLS	StereoSGBM
Block Size	5 Pixels	5 Pixels
Disparity Search Range	96 Pixels	64 Pixels
Confidence Threshold	0.45–0.60	0.50
Processing Resolution	640 × 480	640 × 480
Speech Engine	pyttsx3	pyttsx3
Audio Thread	Asynchronous	Asynchronous
OCR Execution	Background Thread	Periodic Execution
Frame Skip	Real-Time Streaming	5 Frames
OCR Interval	Continuous Background	10 Frames
Power Supply	Dual Li-ion Battery Packs	5 V Power Bank
Main Processor	Raspberry Pi 5	Raspberry Pi 5
Communication	Wi-Fi Streaming	CSI Interface
Stereo Depth Equation	$Z = \frac{fB}{d}$	
Focal Length Used	$f = 700 \text{ pixels}$	
Example Calculation 1	$d = 20 \text{ px} \Rightarrow Z = \frac{700 \times 0.06}{20} = 2.10 \text{ m}$	
Example Calculation 2	$d = 30 \text{ px} \Rightarrow Z = \frac{700 \times 0.06}{30} = 1.40 \text{ m}$	
Example Calculation 3	$d = 40 \text{ px} \Rightarrow Z = \frac{700 \times 0.06}{40} = 1.05 \text{ m}$	
Example Calculation 4	$d = 60 \text{ px} \Rightarrow Z = \frac{700 \times 0.06}{60} = 0.70 \text{ m}$	
Maximum Detectable Classes	80+ Objects (COCO Dataset)	80+ Objects (COCO Dataset)
Text Recognition	Scene Text	Scene Text
Audio Output	Real-Time Guidance	Voice Guidance
Target Application	Indoor/Outdoor Navigation ¹¹	Indoor/Outdoor Navigation



(a) Stereo camera calibration using a chess-board pattern.



(b) YOLO object detection with stereo depth estimation using dual Raspberry Pi cameras.



(c) Wireless stereo image acquisition using dual ESP32-CAM modules.



(d) OCR testing and depth estimation using the stereo vision system.

Figure 5: Experimental validation of the proposed Smart Vision Assistance System. The experiments demonstrate (a) stereo camera calibration and calibration file generation, (b) YOLO-based object detection with stereo depth estimation using dual Raspberry Pi cameras, (c) wireless stereo image acquisition using dual ESP32-CAM modules, and (d) optical character recognition (OCR) integrated with stereo depth estimation.

The experimental results demonstrate the successful implementation of the following functionalities:

- Stereo camera calibration and calibration file generation using a chessboard pattern.
- Synchronized stereo image acquisition using dual Raspberry Pi Camera Modules.
- Wireless stereo image streaming using dual ESP32-CAM modules.
- Stereo disparity map generation using the StereoSGBM algorithm.
- Real-time stereo depth estimation and object distance measurement.
- YOLO-based real-time object detection.

- EasyOCR-based scene text recognition.
- Information fusion combining object detection, depth estimation, and OCR.
- Real-time voice feedback generation for assistive navigation.
- End-to-end validation of both Raspberry Pi Camera and ESP32-CAM based stereo vision systems.

8 Experimental Performance Evaluation

The developed Smart Vision Assistance System was experimentally evaluated using both the Raspberry Pi Camera based wired prototype and the ESP32-CAM based wireless prototype. The evaluation includes object detection, stereo depth estimation, optical character recognition (OCR), wireless communication performance, synchronization accuracy, and overall system latency. All experiments were conducted under indoor laboratory conditions.

8.1 Performance Evaluation of Raspberry Pi Camera Prototype

The Raspberry Pi Camera based prototype was evaluated using dual Raspberry Pi Camera Modules connected through the CSI interface. Table 5 summarizes the observed system performance.

Table 5: Performance Evaluation of Raspberry Pi Camera Prototype

S.No	Performance Metric	Observed Result
1	Object Detection Accuracy (YOLO11n)	92–95%
2	OCR Accuracy	91%
3	System Frame Rate	5 FPS
4	Average Depth Estimation Error	0.35 m
5	Audio Response Delay	1 s
6	End-to-End Processing Time	29 s
7	Processing Lag	14 s (after every 11 s)

Table 1. System Performance Evaluation (Raspberry Pi 5 + Raspberry Pi Camera - Wired Setup)

S.No.	Performance Metric	Observed Result
1	Object Detection Accuracy (YOLO11n)	92–95%
2	OCR Accuracy	91% (minor errors in a few words)
3	System Frame Rate (FPS)	5 FPS
4	Average Depth Estimation Error	0.35 m
5	Audio Response Delay	1 second
6	End-to-End Processing Time	29 seconds
7	Processing Lag During Execution	14 seconds (after approximately every 11 seconds of processing)

Figure 6: Experimental performance evaluation of Raspberry Pi Camera prototype.

8.2 Stereo Depth Estimation Evaluation

The estimated depth was compared with manually measured distances. The absolute depth estimation error was computed using

$$E = |D_{actual} - D_{estimated}| \quad (2)$$

Table 6: Depth Estimation Results

Test	Actual (m)	Estimated (m)	Error (m)
1	0.60	0.90	0.30
2	0.90	1.10	0.20
3	2.00	2.30	0.30
4	0.13	0.60	0.47
5	0.20	0.63	0.43
6	0.30	0.70	0.40
Average Error			0.35

Depth Estimation Test Results			
Test No.	Actual Distance (m)	Measured Distance (m)	Absolute Error (m)
1	0.60	0.90	0.30
2	0.90	1.10	0.20
3	2.00	2.30	0.30
4	0.13	0.60	0.47
5	0.20	0.63	0.43
6	0.30	0.70	0.40
Average	—	—	0.35 m

Figure 7: Experimental stereo depth estimation results.

8.3 OCR Performance Evaluation

OCR accuracy was evaluated using printed book samples.

$$Accuracy = \frac{\text{Correctly Recognized Words}}{\text{Total Words}} \times 100 \quad (3)$$

Table 7: OCR Evaluation Results

Actual Text	Detected Text	Result
Microcontrollers Theory and Application Ajay V. Deshmukh Salient Features For the Book	Microcontrollers Theory and Application Ajay V. Deshmukh Salient Features For hte buuk	Correct Correct Correct Minor Error
Overall OCR Accuracy		91%

OCR Evaluation Samples		
Actual Text	Detected Text	Result
Microcontrollers Theory and Application	Microcontrollers Theory and Application	✓ Correct
Ajay V. Deshmukh	Ajay V. Deshmukh	✓ Correct
Salient Features	Salient Features	✓ Correct
For the Book	For hte buuk	✗ Minor Error
Overall OCR Accuracy = 91%		

Figure 8: OCR evaluation samples.

8.4 ESP32-CAM Wireless Prototype Performance

The wireless stereo vision prototype was evaluated over a Wi-Fi network.

Table 8: ESP32-CAM Wireless Prototype Performance

S.No	Performance Metric	Result
1	Number of ESP32 Modules	2
2	Camera Resolution	640×480
3	Streaming FPS	5 FPS
4	Wireless Latency	0.84 s
5	Average Image Transmission Delay	0.84 s
6	Wi-Fi Range	10–15 m
7	Successful Frame Transmission	96–98%
8	Packet Loss	2–4%
9	Synchronization Difference	0.08 s
10	Power Supply	5 V

Table 2. System Performance Evaluation (Dual ESP32-CAM Wireless Setup)

S.No.	Performance Metric	Observed Result
1	Wireless Communication Latency	0.84 seconds
2	Video Streaming	Stable live video transmission
3	Camera Resolution	640 × 480 pixels
4	Wireless Connectivity	Stable over Wi-Fi during testing
5	Frame Rate	5 FPS (received at Raspberry Pi)
6	Data Transmission Reliability	Stable with occasional network delay

Figure 9: ESP32-CAM wireless prototype performance (Part I).

8.5 Wireless Communication Characteristics

Table 9: Wireless Communication Characteristics

Metric	Description	Observation
Communication Method	Wi-Fi Streaming	Stable
Transmission Latency	Average transmission delay	0.84 s
Synchronization Error	Left-right frame mismatch	0.08 s
Packet Loss	During continuous streaming	2–4%
Frame Delivery Rate	Successfully received frames	96–98%
Operating Range	Indoor testing	10–15 m

Table: Performance Evaluation of Dual ESP32-CAM Wireless Setup

S.No.	Performance Metric	Observed Result
1	Number of ESP32-CAM Modules	2
2	Camera Resolution	640 × 480 pixels (VGA)
3	Frame Rate (Streaming)	5 FPS
4	Wireless Communication Latency	0.84 s
5	Average Image Transmission Delay	0.84 s
6	Wi-Fi Communication Range	≈10–15 m (Indoor)
7	Successful Frame Transmission Rate	96–98%
8	Packet Loss	2–4%
9	Image Synchronization Difference	≈0.08 s
10	Average Power Supply Voltage	5 V

Figure 10: Observed wireless communication characteristics.

8.6 Comparison Between the Two Prototypes

Table 10: Comparison Between Raspberry Pi and ESP32-CAM Prototypes

Parameter	Raspberry Pi	ESP32-CAM
Communication	Wired CSI	Wireless Wi-Fi
Depth Accuracy	High	Moderate
Object Detection Accuracy	92–95%	90–93%
OCR Accuracy	91%	88–91%
Latency	Low	Higher
Streaming Stability	Excellent	Good
Power Consumption	Higher	Lower
Wearability	Pocket Mounted	Smart Glasses
Processing Unit	RPi 5	RPi 5 (Remote)
Best Application	Accurate Navigation	Portable Wearable Vision

9 Current Limitations

Although the proposed Smart Vision Assistance System demonstrates the feasibility of real-time object detection, stereo depth estimation, optical character recognition, and audio-based assistance, several practical limitations remain that should be addressed in future developments.

- The object detection accuracy decreases under poor illumination and low-contrast environments.
- Stereo depth estimation becomes less reliable for transparent, reflective, or texture-less surfaces.
- The current system is limited to object classes available in the pretrained YOLO models and cannot recognize unseen objects.
- OCR performance degrades when text is blurred, partially occluded, rotated, or captured under insufficient lighting.
- Wireless image streaming from ESP32-CAM modules introduces additional communication latency and possible packet loss.
- Continuous execution of deep learning models increases processor temperature and power consumption on the Raspberry Pi 5.
- The current implementation does not perform semantic scene understanding or contextual reasoning between multiple detected objects.
- Audio announcements are generated sequentially and do not yet prioritize obstacles based on user safety.
- Outdoor performance may degrade under rain, fog, or direct sunlight due to camera limitations.
- The proposed wearable prototypes have been designed but are yet to undergo practical fabrication and long-term field evaluation.

10 Future Work

Several enhancements are proposed to improve the functionality, robustness, and usability of the Smart Vision Assistance System.

- Fabrication and experimental validation of both proposed wearable prototypes using 3D-printed enclosures.
- Development of a custom object detection dataset specifically designed for assistive navigation scenarios.
- Integration of GroundingDINO for open-vocabulary object detection.
- Incorporation of Segment Anything Model (SAM) for scene segmentation and object boundary extraction.
- Development of a context-aware decision engine capable of prioritizing important obstacles.
- Integration of temporal memory to avoid repeated announcements of previously detected objects.
- Implementation of intelligent scene understanding using multimodal Vision-Language Models.
- Development of Bluetooth earphone support for improved user comfort.
- Optimization of the software framework for lower computational latency and reduced power consumption.
- Design of a dedicated custom PCB integrating wireless communication, battery management, and embedded processing.
- Integration of GPS, IMU, and electronic compass sensors for outdoor navigation assistance.
- Development of a lightweight mobile application for caregiver monitoring and remote system configuration.

11 Conclusion

This documentation presented the design and implementation of a Smart Vision Assistance System intended to assist visually impaired individuals through real-time environmental perception. The proposed software framework successfully integrates stereo vision, object detection, optical character recognition, depth estimation, and voice feedback within a unified embedded platform.

Two hardware architectures were investigated, namely an ESP32-CAM based wireless stereo vision system and a Raspberry Pi Camera based stereo vision system. Furthermore, two wearable hardware prototypes were proposed to improve portability, usability, and user comfort for future implementation.

The presented system establishes a flexible platform for wearable assistive technology and provides a foundation for future research involving intelligent scene understanding, context-aware navigation, and edge artificial intelligence.

References

1. J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
2. A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” arXiv:2004.10934, 2020.
3. G. Jocher *et al.*, “Ultralytics YOLO Documentation,” Ultralytics Inc., 2024.
4. A. Geiger, P. Lenz, and R. Urtasun, “Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite,” in *Proc. CVPR*, 2012.
5. Z. Zhang, “A Flexible New Technique for Camera Calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000.
6. G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. O’Reilly Media, 2008.
7. OpenCV Development Team, “OpenCV Documentation,” Open Source Computer Vision Library, 2024.
8. Jaied AI, “EasyOCR Documentation,” 2024.
9. Raspberry Pi Foundation, “Raspberry Pi 5 Documentation,” Cambridge, UK, 2024.
10. Raspberry Pi Foundation, “Raspberry Pi Camera Module Documentation,” Cambridge, UK, 2024.
11. Espressif Systems, “ESP32-CAM Technical Reference Manual,” Shanghai, China, 2024.
12. Python Software Foundation, “Python 3 Documentation,” Version 3.12, 2024.
13. K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” arXiv:1409.1556, 2015.
14. D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
15. R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
16. R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
17. S. Garrido-Jurado, R. Muñoz-Salinas, F. Madrid-Cuevas, and M. Marín-Jiménez, “Automatic Generation and Detection of Highly Reliable Fiducial Markers,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014.
18. A. Howard *et al.*, “Searching for MobileNetV3,” in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2019.
19. TensorFlow Lite Team, “TensorFlow Lite Documentation,” Google LLC, 2024.

20. P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*, 2nd ed. Springer, 2017.